

1. System Overview

The project aims to design and implement a **Retrieval-Augmented Generation (RAG)-based Customer Support Assistant**.

The system allows users to query a knowledge base (PDF document) and receive accurate, context-aware responses. It combines **information retrieval techniques with Large Language Models (LLMs)** to generate meaningful answers.

Additionally, the system includes a **Human-in-the-Loop (HITL)** mechanism to handle low-confidence queries by escalating them to a human agent.

2. Problem Statement

Traditional customer support systems:

- Fail to provide contextual answers
- Require manual intervention
- Do not scale efficiently

This system solves these issues by:

- Automating responses using RAG
- Retrieving relevant document context
- Escalating complex queries intelligently

3. Scope of the System

The system supports:

- PDF-based knowledge ingestion
- Contextual query answering
- Graph-based workflow execution
- Conditional routing
- Human escalation

Limitations:

- Works on single document (can be extended)
- Limited reasoning capability due to lightweight model

4. Architecture Diagram (WRITE THIS IN WORD USING BLOCKS)

User



Query Input



LangGraph Workflow



[Processing Node]



Retriever (Embeddings + Similarity Search)



LLM (Text Generation)



Decision Layer



Answer Escalation (HITL)



Human Response

5. System Components

1. Document Loader

Loads PDF file using PyPDF and extracts raw text.

2. Chunking Module

Splits text into smaller chunks (500 characters) for better retrieval.

3. Embedding Model

Uses Sentence Transformers to convert text into vector representations.

4. Vector Storage

Stores embeddings in memory (can be extended to ChromaDB).

5. Retriever

Finds most relevant chunks using similarity search.

6. LLM (Language Model)

Generates answers using context (tiny GPT model).

7. Graph Workflow Engine (LangGraph)

Controls flow of execution using nodes and edges.

8. Routing Layer

Determines whether to:

- Answer automatically
- Escalate to human

9. HITL Module

Handles human intervention for low-confidence queries.

6. Data Flow

1. PDF is loaded and converted into text
2. Text is split into chunks
3. Each chunk is converted into embeddings
4. User enters query
5. Query is converted into embedding
6. Similar chunks are retrieved
7. Context is passed to LLM
8. LLM generates response
9. If confidence is low → escalate to human

7. Technology Choices

- **Python** → Easy development
- **PyPDF** → PDF processing
- **Sentence Transformers** → Efficient embeddings
- **NumPy** → Fast similarity computation
- **Transformers (HuggingFace)** → LLM
- **LangGraph** → Workflow orchestration

8. Scalability Considerations

- Can replace in-memory storage with ChromaDB
- Can support multiple PDFs
- Can integrate APIs for real-time queries
- Can deploy on cloud platforms